01 — Master Project Development Guide

Project: PriceRadar — Real-Time Product Price Comparison and Intelligent Price-Drop Alert System **Status:** Authoritative source of truth. Every other document in `docs/` must match this one. If any other document disagrees with this file, this file wins and the other document is wrong and must be fixed. **Team:** 4 beginner developers, 7 calendar days, Windows 10/11 + VS Code + PowerShell.

---

1. Project overview and scope

PriceRadar lets a user search for a product, see its price across more than one retailer side by side, save offers to a personal watchlist, set an alert ("tell me when this hits ₹X" or "tell me when this drops by ₹Y"), and get notified by email and/or Telegram when that condition is met. Monitoring runs on the server on a schedule — never in the user's browser.

1.1 In scope for the 7-day MVP

- Search and compare offers for the same product across at least two labeled sources (see Section 10 — one real source if approved in time, one clearly labeled sample source; see Section 10.4 for why).
- Registration, login, JWT-based auth, per-user private watchlist and alerts.
- Two alert types: target-price and fixed-amount drop, with duplicate suppression and re-arming.
- Email (Brevo) and Telegram notifications, with delivery history.
- A scheduled, server-side monitoring job (not a browser timer) that fetches each distinct offer once per cycle and evaluates every alert that depends on it.
- A protected endpoint to trigger a monitoring cycle on demand, for demos and testing.
- Deployment of frontend, backend, and database, with a working health check.

1.2 Out of scope for the 7-day MVP

Per the project brief, explicitly excluded: automatic purchasing, AI price predictions, microservices, a payment system, browser-based scraping workarounds, CAPTCHA bypasses, and any claim of live retailer access that hasn't been verified. Price-history charts are a stretch goal (Day 6/7 only, never at the expense of the core flow).

1.3 Non-functional requirements

Requirement	What it means here
Honesty	The UI must never present sample/simulated prices as live. Every offer card shows its source label.
Data isolation	A user can only ever read or write their own watchlist, alerts, and notification history.
Resilience	A failed scrape/API call never becomes a price of ₹0 and never fires a false alert.
No duplicate spam	An unchanged alert condition does not notify again on every cycle.
Secrets hygiene	No credential ever appears in a commit, a log line, or frontend code.
Beginner-maintainability	No feature is implemented in a way that only one team member understands.

---

2. Architecture and data flow

```
Browser (React + Vite)
         |  HTTPS (fetch/axios, JWT in Authorization header)
         v
  Express API (Node.js) ---- Mongoose ----> MongoDB Atlas
         ^
         |  POST /api/monitor/run (shared-secret header, not a user JWT)
  GitHub Actions scheduled workflow (runs every hour, see Section 15)
         |
         v
  Provider layer (server/src/providers/) --- HTTPS ---> Flipkart Affiliate API (if approved)
                                                    -> Fixture provider (sample data, always available)
         |
         v
  Notification services (server/src/services/notifications/) --- HTTPS ---> Brevo API (email)
                                                                        -> Telegram Bot API
```

Request flow for a normal page view: Browser → Express route → controller → service/model → MongoDB → JSON response → React renders it.

Request flow for monitoring: GitHub Actions wakes the API on a schedule → `POST /api/monitor/run` → `priceMonitor.js` loads every **active, distinct** offer that's on at least one watchlist or alert → calls the matching provider once per offer → writes a `PriceObservation` → `alertEvaluator.js` checks every alert tied to that offer → on a trigger, calls the notification service → a `NotificationEvent` is recorded either way (sent, failed, or skipped-duplicate).

The key design rule baked into this: **one offer is fetched once per cycle, no matter how many users are tracking it.** The monitor loops over distinct offers, not over users or alerts.

---

3. Technology stack (final)

Layer	Choice	Why
Frontend	React 18 + Vite + React Router v6	Fast dev server, the version the whole team will have in `packag e.json`, beginner-friendly routing
Styling	Bootstrap 5 via CDN `<link>` / `<script>` in `index.html`, plus one custom stylesheet `client/src/styles/theme.css`	No extra npm build-tool complexity; consistent components (cards, badges, forms) out of the box
HTTP client	Axios	Clearer error objects than raw `fetch`; used everywhere, consistently (not mixed with `fetch`)
Backend	Node.js 20 LTS + Express 4	Standard, stable, huge amount of beginner documentation
Database	MongoDB Atlas (free M0 cluster) + Mongoose 8	Free, managed, no local DB install needed, works identically for all 4 members
Auth	bcryptjs (hashing) + jsonwebtoken (JWT)	No native build step (bcryptjs is pure JS, avoids Windows node-gyp pain); JWT is simple to reason about for 4 beginners
Validation	express-validator	Declarative, keeps controllers thin
Retailer data	Axios + Cheerio for provider calls; see Section 10 for which sources are real vs. fixture	Lightweight, no headless browser needed for the API/HTML the approved source returns
Scheduler	GitHub Actions scheduled workflow (`cron:`) calling a protected endpoint	The only genuinely free, reliable "every hour" option — see Section 15 for why a hosted cron/worker is not used
Email	Brevo transactional email API	Verified free plan: 300 emails/day, no time limit (Section 11)
Messaging	Telegram Bot API (raw HTTPS calls via Axios)	Free, no approval process, official API
Testing	Jest + Supertest (backend), manual Postman collection (frontend-facing)	One consistent framework the whole backend side uses
Tooling	VS Code, Git, GitHub, Postman, PowerShell	As specified for the team

Node version: Node.js 20 LTS. Check what's installed with:

```
node --version
  npm --version
```

If `node --version` prints anything below `v20`, install Node 20 LTS from nodejs.org before Day 1 setup.

Do not add a package that isn't in this table without updating this document first — an unplanned dependency on one member's machine is a common source of "works on my machine" bugs.

---

4. Repository structure (final)

```
PriceRadar/
  ├── client/
  │   ├── public/
  │   ├── src/
  │   │   ├── components/        # reusable UI pieces (OfferCard, AlertForm, Navbar, ...)
  │   │   ├── pages/             # one file per route (SearchPage.jsx, DashboardPage.jsx, ...)
  │   │   ├── services/          # api.js — the one shared Axios client
  │   │   ├── context/            # AuthContext.jsx
  │   │   ├── styles/             # theme.css
  │   │   ├── App.jsx
  │   │   └── main.jsx
  │   ├── index.html
  │   ├── package.json
  │   └── .env.example
  ├── server/
  │   ├── src/
  │   │   ├── config/             # db.js (Mongoose connection)
  │   │   ├── models/             # User.js, Product.js, Offer.js, PriceObservation.js, WatchlistItem.js, Alert.j
  s, NotificationEvent.js
  │   │   ├── routes/             # auth.js, products.js, offers.js, watchlist.js, alerts.js, notifications.js, mo
  nitor.js
  │   │   ├── controllers/        # one controller file per route file
  │   │   ├── middleware/         # auth.js (JWT check), errorHandler.js, validate.js
  │   │   ├── services/
  │   │   │   ├── priceComparison.js   # owned by Member 1
  │   │   │   ├── priceMonitor.js      # owned by Member 1
  │   │   │   ├── alertEvaluator.js    # owned by Member 1, coordinate with Member 3
  │   │   │   └── notifications/       # email.js, telegram.js — owned by Member 4
  │   │   ├── providers/           # flipkart.js, fixture.js, index.js (the dispatcher) — owned by Member 4
  │   │   ├── jobs/                # (kept for local dev convenience only — see Section 15) — owned by Member 4
  │   │   ├── utils/               # money.js (paise helpers), asyncHandler.js
  │   │   ├── app.js               # Express app, middleware wiring, route mounting
  │   │   └── server.js            # starts the HTTP server, calls connectDB()
  │   ├── tests/
  │   ├── package.json
  │   └── .env.example
  ├── docs/
  │   ├── 01-MASTER-PROJECT-DEVELOPMENT-GUIDE.md
  │   ├── 02-FRONTEND-AND-API-INTEGRATION-GUIDE.md
  │   ├── 03-BACKEND-DATABASE-AND-AUTHENTICATION-GUIDE.md
  │   ├── 04-RETAILER-DATA-SCHEDULER-AND-NOTIFICATIONS-GUIDE.md
  │   └── 05-TEAM-LEAD-GITHUB-INTEGRATION-AND-DEPLOYMENT-GUIDE.md
  ├── .github/
  │   └── workflows/
  │       └── monitor.yml          # the GitHub Actions scheduled trigger — see Section 15
  ├── .gitignore
  └── README.md
```

This structure is final. If it must change, update this file first, then every document that references a path.

5. Database schema (final)

Precision policy: every price is stored as an **integer number of paise** (`price × 100`, rounded) in a field named `priceInPaise`. Never store money as a JavaScript float. Convert to rupees only for display: `priceInPaise / 100`. This avoids classic floating-point bugs (`0.1 + 0.2 !== 0.3`) in price-drop math. `currency` is stored alongside every price and is always `"INR"` for this MVP; a field exists so the system is honest about not silently treating other currencies as equivalent (see Section 1.1 — no cross-currency comparison is implemented).

5.1 `users`

Field	Type	Required	Notes
`_id`	ObjectId	auto	
`name`	String	yes	
`email`	String	yes	unique index, lowercase, validated format
`passwordHash`	String	yes	bcryptjs hash, never returned in any API response
`telegramChatId`	String	no	set via the Telegram link flow, Section 8
`createdAt` / `updatedAt`	Date	auto	Mongoose timestamps

Owner: Member 3. Access: a user can only read/update their own document (enforced by the `req.user.id` from the JWT, never a client-supplied id).
Example document:

```
{
    "_id": "670000000000000000000a1",
    "name": "Priya Sharma",
    "email": "priya@example.com",
    "passwordHash": "$2a$10$...",
    "telegramChatId": "123456789",
    "createdAt": "2026-10-13T06:00:00.000Z"
  }
```

5.2 `products`

The **canonical product identity** — what a search result groups offers under. Distinct from an `Offer`, which is one retailer's listing of it.

Field	Type	Required	Notes
`_id`	ObjectId	auto	
`title`	String	yes	e.g. "Samsung Galaxy M14 5G"
`brand`	String	no	
`model`	String	no	
`variant`	String	no	e.g. "128GB, Ice Blue" — variant-defining attributes live here so a 128GB and 256GB phone are never merged
`canonicalKey`	String	yes	unique index; a normalized slug (`brand-model-variant`, lowercased, hyphenated) used to prevent duplicate product documents for the same real-world item
`imageUrl`	String	no	
`createdAt`	Date	auto	

Owner: Member 4 (created by the provider normalization step) with model file owned by Member 3.

5.3 `offers`

One retailer's listing of a product. This is the unit the monitor fetches and the unit a user actually watches/alerts on — never the abstract `Product`.

Field	Type	Required	Notes
`_id`	ObjectId	auto	
`productId`	ObjectId → `products`	yes	
`provider`	String enum `["flipkart", "amazon"]`	yes	`"amazon"` only ever appears with `isSample: true` — see Section 10.3
`externalProductId`	String	yes	the ID the provider itself uses; unique together with `provider` (compound unique index `{provider, externalProductId}`) so the same real listing is never duplicated
`title`	String	yes	the title as returned by the source (may differ slightly from the canonical product title)
`productUrl`	String	yes	
`priceInPaise`	Number (integer)	yes	the last **successful** price; never set to 0 on a failed fetch
`currency`	String	yes	`"INR"`
`availability`	String enum `["in_stock", "out_of_stock", "unknown"]`	yes	
`isSample`	Boolean	yes	`true` for fixture/seeded data; `false` only for a verified live source. Must be shown in the UI whenever `true`.
`lastFetchedAt`	Date	no	timestamp of the last **attempt** (success or failure)
`lastSuccessAt`	Date	no	timestamp of the last **successful** observation
`lastStatus`	String enum `["ok", "stale", "error"]`	yes, default `"stale"`	`"stale"` = no successful fetch yet or the offer hasn't been re-checked recently; `"error"` = the last attempt failed; `"ok"` = last attempt succeeded
`lastError`	String	no	human-readable reason for the most recent failure, cleared on the next success
`createdAt` / `updatedAt`	Date	auto	

Owner: Member 4 (provider writes), Member 3 (model + read routes).

5.4 `priceobservations`

The append-only history log behind every price-drop calculation and the stretch-goal chart.

Field	Type	Required	Notes
`_id`	ObjectId	auto	
`offerId`	ObjectId → `offers`	yes	indexed
`priceInPaise`	Number	yes, only present when `status: "success"`	
`status`	String enum `["success", "failure"]`	yes	
`errorMessage`	String	no	only when `status: "failure"`
`fetchedAt`	Date	yes, default now	indexed together with `offerId` for fast "most recent observation" queries

Owner: Member 4 (writer), Member 1 (reader, for comparison/alert logic).

5.5 `watchlistitems`

Field	Type	Required	Notes
`_id`	ObjectId	auto	
`userId`	ObjectId → `users`	yes	indexed
`offerId`	ObjectId → `offers`	yes	
`createdAt`	Date	auto	

Compound unique index `{userId, offerId}` — a user cannot watch the same offer twice. Owner: Member 3.

5.6 `alerts`

Field	Type	Required	Notes
`_id`	ObjectId	auto	
`userId`	ObjectId → `users`	yes	indexed
`offerId`	ObjectId → `offers`	yes	
`type`	String enum `["target_price", "fixed_drop"]`	yes	
`targetPriceInPaise`	Number	required when `type === "target_price"`	
`dropAmountInPaise`	Number	required when `type === "fixed_drop"`	
`channels`	Array of String enum `["email", "telegram"]`	yes, min length 1	
`status`	String enum `["active", "paused"]`	yes, default `"active"`	
`armed`	Boolean	yes, default `true`	see re-arming rule, Section 9
`lastTriggeredAt`	Date	no	
`createdAt` / `updatedAt`	Date	auto	

Owner: Member 3 (CRUD routes/model), Member 1 (`alertEvaluator.js` reads/updates `armed` and `lastTriggeredAt` — coordinate before changing this model).

5.7 `notificationevents`

Field	Type	Required	Notes
`_id`	ObjectId	auto	
`userId`	ObjectId → `users`	yes	indexed
`alertId`	ObjectId → `alerts`	yes	
`channel`	String enum `["email", "telegram"]`	yes	
`status`	String enum `["sent", "failed", "skipped_duplicate"]`	yes	
`message`	String	yes	the actual text sent (or that would have been sent)
`errorMessage`	String	no	
`dedupeKey`	String	yes	see Section 9.3; unique index `{alertId, dedupeKey}`
`createdAt`	Date	auto	

Owner: Member 4.

5.8 Distinctions the schema encodes

Concept	How it's represented
Canonical product vs. retailer offer	`Product` vs. `Offer` — separate collections
Successful observation vs. failed fetch	`PriceObservation.status`
Stale offer	`Offer.lastStatus === "stale"` (no fetch yet / overdue)
Fetch error	`Offer.lastStatus === "error"`, with `Offer.lastError` and a matching `PriceObservation.status === "failure"` row
Alert condition (the rule)	The `Alert` document itself
Alert event (a trigger firing)	`Alert.lastTriggeredAt` updated + one `NotificationEvent` per channel
Notification attempt	`NotificationEvent`, one row per channel per trigger, regardless of outcome

---

6. REST API contract (final)

Base path: `/api`. **Auth:** `Authorization: Bearer <jwt>` header, required on every route marked "Yes" below, except `/api/monitor/run` which uses a separate shared-secret header (never a user JWT — see Section 6.3).

Success envelope: `{ "success": true, "data": <payload> }` **Error envelope:** `{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "Human-readable explanation." } }`

Error codes used: `VALIDATION_ERROR` (400), `AUTH_REQUIRED` (401), `AUTH_INVALID` (401), `FORBIDDEN` (403), `NOT_FOUND` (404), `CONFLICT` (409, e.g. duplicate email or duplicate watchlist item), `SERVER_ERROR` (500).

6.1 Auth

Method	Path	Auth	Body	Success	Status
POST	`/api/auth/register`	No	`{name, email, password}`	`{user: {id,name,email}}`	201
POST	`/api/auth/login`	No	`{email, password}`	`{token, user: {id,name,email}}`	200
GET	`/api/auth/me`	Yes	—	`{user: {id,name,email,telegramChatId}}`	200

6.2 Products and offers

Method	Path	Auth	Params	Success	Status
GET	`/api/products/search`	No	`?q=<text>`	`{products: [{id, title, brand, variant, offers: [OfferSummary]}]}`	200
GET	`/api/products/:productId`	No	—	`{product, offers: [OfferSummary]}`	200
GET	`/api/offers/:offerId/history`	No	—	`{offerId, history: [{priceInPaise, status, fetchedAt}]}`	200

`OfferSummary` shape: `{id, provider, title, priceInPaise, currency, availability, productUrl, isSample, lastStatus, lastSuccessAt}`.

6.3 Watchlist

Method	Path	Auth	Body	Success	Status
GET	`/api/watchlist`	Yes	—	`{items: [{id, offer: OfferSummary, createdAt}]}`	200
POST	`/api/watchlist`	Yes	`{offerId}`	`{item}`	201
DELETE	`/api/watchlist/:watchlistItemId`	Yes	—	`{deleted: true}`	200

6.4 Alerts

Method	Path	Auth	Body	Success	Status
GET	`/api/alerts`	Yes	—	`{alerts: [Alert]}`	200
POST	`/api/alerts`	Yes	`{offerId, type, targetPriceInPaise?, dropAmountInPaise?, channels}`	`{alert}`	201
PATCH	`/api/alerts/:alertId`	Yes	any subset of `{status, targetPriceInPaise, dropAmountInPaise, channels}`	`{alert}`	200
DELETE	`/api/alerts/:alertId`	Yes	—	`{deleted: true}`	200

6.5 Notifications

Method	Path	Auth	Body	Success	Status
GET	`/api/notifications`	Yes	—	`{events: [NotificationEvent]}`	200
GET	`/api/notification-preferences`	Yes	—	`{email, telegramChatId, telegramLinked: boolean}`	200
PUT	`/api/notification-preferences`	Yes	`{email?}`	`{email, telegramChatId, telegramLinked}`	200
POST	`/api/telegram/link-code`	Yes	—	`{linkCode, botUsername, expiresAt}`	200
POST	`/api/telegram/link`	Yes	`{linkCode}`	`{linked: true}`	200

`POST /api/telegram/link-code` was added to this contract on 2026-10-14, folded in from a gap Member 4 flagged while building the Telegram integration (Document 4, Section 12) and Member 3 flagged independently while building the route shell (Document 3, Section 11). This is the first half of the linking flow: the frontend calls this (no body, just the caller's JWT) to generate a short-lived code tied to `req.user.id`, which the user then sends to the bot as `/start <linkCode>` before the frontend calls `POST /api/telegram/link` with that same code to complete the link. `linkCode` expires at `expiresAt` (ISO 8601) if never used. This is the authoritative addition — Documents 3 and 4 already describe it exactly this way; no further change is needed in either.
Full Telegram linking flow: the user, logged in, opens Notification Preferences and clicks "Link Telegram" → frontend calls `POST /api/telegram/link-code` → user sends `/start <linkCode>` to the bot in Telegram → the backend's Telegram listener (owned by Member 4, Document 4 Section 12) matches the code to the `userId` and records `User.telegramChatId` → frontend calls `POST /api/telegram/link` with the same code to confirm the link client-side and refresh `telegramLinked`. See Document 4 for the exact bot-polling mechanism.

6.6 Monitoring (protected, not a user route)

Method	Path	Auth	Body	Success	Status
GET	`/api/health`	No	—	`{status: "ok", time: "<ISO8601>"}`	200
POST	`/api/monitor/run`	`X-Monitor-Secret` **header must equal** `MONITOR_TRIGGER_SECRET`	—	`{started: true, offersChecked: n, triggeredAlerts: n}`	202
GET	`/api/monitor/status`	Yes (any logged-in team member)	—	`{lastRunAt, lastRunOffersChecked, lastRunErrors}`	200

`/api/monitor/run` deliberately does **not** use the normal user JWT — it's called by the GitHub Actions workflow, not by a logged-in user, and must not be triggerable by any visitor who can log in. A wrong or missing `X-Monitor-Secret` returns `401 AUTH_INVALID`.

6.7 Example request/response

```
POST /api/alerts
  Authorization: Bearer eyJhbGciOi...
  Content-Type: application/json

  {
    "offerId": "670000000000000000000b2",
    "type": "target_price",
    "targetPriceInPaise": 1499900,
    "channels": ["email", "telegram"]
  }
```

```
{
    "success": true,
    "data": {
      "alert": {
        "id": "670000000000000000000c3",
        "offerId": "670000000000000000000b2",
        "type": "target_price",
        "targetPriceInPaise": 1499900,
        "channels": ["email", "telegram"],
        "status": "active",
        "armed": true,
        "createdAt": "2026-10-13T06:10:00.000Z"
      }
    }
  }
```

Never return `passwordHash`, `MONITOR_TRIGGER_SECRET`, or any provider token in any response.

---

7. Environment variables (final)

7.1 `server/.env`

Variable	Required	Secret	Used by	Dev example
`PORT`	yes	no	server.js	`5000`
`NODE_ENV`	yes	no	app.js	`development`
`MONGODB_URI`	yes	**yes**	config/db.js	`mongodb+srv://user:pass@cluster0.mongodb.net/priceradar`
`JWT_SECRET`	yes	**yes**	middleware/auth.js	a long random string, e.g. generate with `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`
`JWT_EXPIRES_IN`	yes	no	controllers/auth.js	`7d`
`CLIENT_ORIGIN`	yes	no	app.js (CORS)	`http://localhost:5173`
`FLIPKART_AFFILIATE_ID`	no	**yes**	providers/flipkart.js	*(blank until approved — see Section 10)*
`FLIPKART_AFFILIATE_TOKEN`	no	**yes**	providers/flipkart.js	*(blank until approved)*
`BREVO_API_KEY`	yes	**yes**	services/notifications/email.js	from Brevo dashboard
`EMAIL_FROM`	yes	no	services/notifications/email.js	a verified sender address, e.g. `alerts@yourteamdomain.example`
`TELEGRAM_BOT_TOKEN`	yes	**yes**	services/notifications/telegram.js	from @BotFather
`MONITOR_TRIGGER_SECRET`	yes	**yes**	routes/monitor.js	a long random string, same generation method as JWT_SECRET
`MONITORING_ENABLED`	yes	no	services/priceMonitor.js	`true`

7.2 `client/.env`

Variable	Required	Secret	Used by	Dev example
`VITE_API_URL`	yes	no	services/api.js	`http://localhost:5000`

No other variable is ever readable from the frontend. Vite only exposes variables prefixed `VITE_`, which is itself a safety net — nothing secret may ever be given that prefix.

7.3 Rules

- `.env` is never committed. `server/.gitignore` and `client/.gitignore` both list `.env`.
- `server/.env.example` and `client/.env.example` are committed, with placeholder values only (shown above).
- If a secret is ever accidentally committed: remove it from the file, commit the removal, **then rotate the actual credential** (generate a new Mongo password / JWT secret / API key) — deleting it from a later commit does not remove it from git history.
- In production (Section 15), these same variables are set in the hosting platform's dashboard, never in a committed file.

---

8. Security rules

- Passwords are hashed with bcryptjs (`bcrypt.hash(password, 10)`) before storage. Plaintext passwords are never logged or stored.
- JWT is stored in the browser in `localStorage` under the key `priceradar_token`. **This is a deliberate, documented trade-off, not a claim that it's maximally secure:** `localStorage` is readable by any JavaScript running on the page, so it is vulnerable if the app ever has an XSS bug. The alternative (an httpOnly cookie) needs CSRF protection and cross-origin cookie

configuration, which is a lot of additional surface area for 4 beginners in 7 days. The mitigation is to never render unescaped user input (React escapes by default — don't use `dangerouslySetInnerHTML`) and to keep the JWT's lifetime short (`JWT_EXPIRES_IN=7d` is already a compromise; do not extend it).

- Every route under `/api/watchlist`, `/api/alerts`, `/api/notifications`, `/api/notification-preferences` loads data scoped to `req.user.id` (from the verified JWT) — **never** from a client-supplied `userId` in the body or query string. A controller that does `Alert.find({ userId: req.body.userId })` is a bug; it must be `Alert.find({ userId: req.user.id })`.
- `/api/monitor/run` is the one route that is intentionally *not* behind user auth — it's behind a separate shared secret because it isn't called by a user at all (Section 6.6).
- CORS: `app.use(cors({ origin: process.env.CLIENT_ORIGIN }))` — never `origin: "*"` once real user JWTs are involved.
- MongoDB Atlas network access: during development, "Allow access from anywhere" (`0.0.0.0/0`) is acceptable since the cluster still requires a username/password. Before the final demo, note in the deployment doc (Document 5) that this is a deliberate simplification for a 7-day training project, not a production-grade setup.
- Validation: every route that accepts a body uses `express-validator` rules in `middleware/validate.js`, and a request that fails validation never reaches a controller.
- Centralized error handling: every thrown error passes through one `errorHandler.js` middleware that formats the error envelope (Section 6) and never leaks a stack trace or internal file path to the client in `NODE_ENV=production`.

---

9. Alert logic (final, authoritative — Document 4 must match this exactly)

9.1 Target-price alert

Triggers when a **successful** `PriceObservation` for the alert's `offerId` has `priceInPaise <= alert.targetPriceInPaise`.

9.2 Fixed-drop alert

```
priceDrop = previousSuccessfulPriceInPaise - currentSuccessfulPriceInPaise
  trigger when priceDrop >= alert.dropAmountInPaise
```

`previousSuccessfulPriceInPaise` is the `priceInPaise` of the **immediately preceding** `PriceObservation` with `status: "success"` for the same `offerId` (not an average, not the all-time high — exactly the one before this one). If there is no prior successful observation yet, the alert cannot trigger on this cycle (there's nothing to compare against) — this is normal and expected on an offer's first successful check.

9.3 Duplicate suppression and re-arming

Each `Alert` carries an `armed` boolean, default `true`.

- A trigger only fires a notification when the alert is currently `armed: true`.
- On a successful trigger: send the notification(s), set `armed: false`, set `lastTriggeredAt: now`. This stops the same still-true condition from notifying again every hour.
- **Re-arming rule:** on a later cycle, if a fresh successful observation shows the condition is **no longer true** (price has risen back above target, for target-price; or the drop versus the *new* previous observation no longer meets the threshold, for fixed-drop), set `armed: true` again. The alert is now eligible to trigger again the next time the condition becomes true.
- `dedupeKey` on `NotificationEvent` is built as `${alertId}:${offerId}:${priceInPaise}:${triggeredAtHour}` — this is a second, independent safety net: even if the evaluator somehow runs twice for the same hour (e.g. a manual `/api/monitor/run` plus the scheduled one landing in the same hour), the unique index `{alertId, dedupeKey}` on `NotificationEvent` means the second attempt is recorded as `status: "skipped_duplicate"` rather than sending a second message.
- A failed `PriceObservation` (`status: "failure"`) never changes `armed` and never triggers an alert.

9.4 Walkthrough example

Target price ₹14,999 (`1499900` paise) on an offer currently ₹15,499:

1. Hour 1: price ₹15,499 → no trigger, `armed` stays `true`.
2. Hour 2: price drops to ₹14,799 → condition true, `armed` is `true` → notify, set `armed: false`.

3. Hour 3: price ₹14,799 (unchanged) → condition still true, but `armed` is `false` → no second notification.
4. Hour 4: price rises to ₹15,199 → condition false → re-arm: `armed: true`.
5. Hour 5: price drops to ₹14,999 → condition true again, `armed` is `true` → notify again.

---

10. Provider strategy and Day-1 feasibility findings

This section was investigated and verified on **2026-10-13** against each provider's official documentation. It is the single most important risk in the whole project, which is why the brief puts it on Day 1.

10.1 Flipkart Affiliate API — the real-data candidate

Verified from the official docs (Overview, Registration):

- The Product/Offer/Report APIs (JSON and XML over HTTPS) are available **only to registered, approved Flipkart affiliates.** There is no unauthenticated or instantly-self-serve tier.
- To get credentials: register at `affiliate.flipkart.com/registerme` → once your account is **approved**, go to API → API Token → Generate API Token → you receive an Affiliate Tracking ID + API Token, sent as `Fk-Affiliate-Id` / `Fk-Affiliate-Token` headers.
- Registration requires a live "site" (the affiliate program is built around having a website/app to link from — your deployed PriceRadar frontend qualifies) and KYC details (PAN, per third-party guidance; official policy requires accurate registration information).
- **Approval is not instant.** Third-party sources researched alongside the official docs report approval windows ranging from a few days to a few weeks. This is **unverified for certain** as an exact number — Flipkart's own docs don't publish an SLA — but every independent source agrees it is not same-day.
- There is a separate, different "Flipkart Marketplace Seller API" for sellers managing their own listings — **this is not the right API for a price-comparison tool,** and Flipkart's own Seller Hub terms explicitly restrict automated/third-party access to it and forbid sharing seller credentials with aggregators. Do not use the Seller API.

Decision: Member 4 registers for the Flipkart Affiliate Program on Day 1, **the moment the team has a live (even placeholder) deployed URL to register with.** This is a race against the 7-day clock that may or may not finish in time. Build against a fixture provider from Day 1 regardless (Section 10.3), and swap in the real Flipkart provider the moment — and only the moment — approval and a token actually arrive, verified by a real, logged API response, not an assumption.

10.2 Amazon India Creators API — not usable for this team

Verified from the official docs (Introduction):

> *"Prerequisites... Have at least **10 qualifying sales within the past 30 days** to access the PA API through the Creators API."*

This is a hard, documented eligibility gate. A brand-new team with a brand-new Associates account has zero sales history and **cannot** reach 10 qualifying sales in 30 days from a standing start inside a 7-day project — this isn't a matter of applying faster, it's a structural requirement that doesn't fit the timeline at all.

Decision, per the brief's own instruction not to make Amazon a critical dependency: Amazon is **excluded from real-data sourcing entirely** for this project. If Amazon appears in the UI at all, it is only as a second `isSample: true` fixture offer, purely to make the comparison view show more than one retailer for demo purposes, and it is labeled "Sample data" everywhere it's shown. No code path ever attempts a live Amazon API call.

10.3 Fixture / seeded provider — the guaranteed fallback

`server/src/providers/fixture.js` returns a small, hand-written set of realistic products and offers (5–8 products, both `flipkart`- and `amazon`-labeled entries, `isSample: true` on all of them) with prices that can be edited in a JSON file to **demonstrate a price drop on demand** during the live demo (Day 7) without waiting for a real price to actually change. This is not a cop-out — the brief explicitly requires exactly this honest fallback (Section 10.5 of the brief) — and it's also simply the only way to reliably demo a "price dropped, alert fired" moment in a 7-day window regardless of what happens with Flipkart approval.

10.4 Why the UI always shows at least two labeled sources

Even in the best case (Flipkart approved in time), real Flipkart data alone is one retailer, which doesn't demonstrate "comparison." The fixture provider's `amazon`-labeled, clearly-sample entries exist specifically to make the comparison UI meaningful without pretending to have live Amazon access. **If and when** Flipkart approval lands, Flipkart's `isSample` flips to `false` for those offers; Amazon's `isSample` stays `true` forever in this project.

10.5 Provider interface (final)

Every provider — real or fixture — implements the same function signature and returns the same normalized shape, so `priceMonitor.js` never knows or cares which provider it's calling:

```
// server/src/providers/index.js — the dispatcher
  // providers/<name>.js each export: async function fetchOffer(offer) -> NormalizedResult

  // NormalizedResult shape (always):
  {
    provider: "flipkart" | "amazon",
    externalProductId: string,
    title: string,
    brand: string | null,
    model: string | null,
    variant: string | null,
    priceInPaise: number,       // integer, only present on success
    currency: "INR",
    availability: "in_stock" | "out_of_stock" | "unknown",
    productUrl: string,
    imageUrl: string | null,
    isSample: boolean,
    fetchedAt: string,          // ISO 8601, set by the caller, not the provider
    success: boolean,
    errorMessage: string | null // present when success === false
  }
```

A provider function **never throws** for an ordinary fetch failure (network error, item not found, rate-limited) — it catches internally and returns `{ success: false, errorMessage: "..." }` so `priceMonitor.js` has one consistent way to handle every failure without a `try/catch` around every provider call.

10.6 Honest fallback policy, applied

Per the brief's required policy: this document records which source was tested (Flipkart Affiliate API, Amazon Creators API), what prevented unconditional access (approval latency for Flipkart; a 10-sale eligibility gate for Amazon, which cannot be met by a new account), and commits to building and testing the entire system against the fixture provider from Day 1, swapping in real Flipkart data only if and when it is actually, verifiably approved. The final demo (Day 7) will state plainly which offers are live and which are labeled sample data. A working demo on fixture data is the acceptance bar this project commits to; live Flipkart data is a bonus, not a requirement for "done."

---

11. Email provider: Brevo — verified free-plan limits

Verified from Brevo's official help article, checked 2026-10-13:

- Free plan: **300 email sends per day**, resets daily, unused sends do not roll over. No time limit on the free plan itself.
- Transactional emails beyond the daily cap are queued (up to 1,000) and sent as the quota allows — they are not simply dropped, but at 300/day for a 4-person team demo with a handful of test alerts, the team will not come close to this limit.
- Up to 100,000 stored contacts; only 1 user per free account (so only Member 4's or the team's shared account should hold the Brevo login — do not create 4 separate free Brevo accounts for one project, since each has its own daily cap and that fragments delivery history unnecessarily. Use one shared account; see Document 4 for how to store the single `BREVO_API_KEY` without exposing it).

- Sender verification is required by Brevo for deliverability (a "From" address/domain must be verified in the Brevo dashboard before sending) — this is a Day-1/Day-2 setup step for Member 4, documented in Document 4.

Decision: Brevo's free plan comfortably covers this project. No paid plan or alternative provider is needed.

---

12. Deployment architecture and free-tier findings (summary — full procedure in Document 5)

Verified 2026-10-13 against current hosting-provider documentation and pricing pages. Full citations and step-by-step deployment are in Document 5; this section records the decision and why.

- **Railway** no longer has a persistent free tier: new accounts get a one-time $5 usage credit (no renewing free allotment), after which it bills per second of usage. For a project that must run the full 7 days plus the demo, this is a real risk of the trial credit running out mid-week with no replacement ("Railway reverts to a Free plan at $1/month equivalent credit with hard resource caps, not enough for a real multi-service app").
- **Render** has a genuinely perpetual free tier for **web services**, but a free web service **sleeps after 15 minutes of no inbound traffic** and takes roughly a minute to wake on the next request. Render has **no free instance type for a dedicated Background Worker** (the "always-on" service type) — the cheapest worker is a paid instance. A Render Cron Job service type exists and can run on a schedule, billed per minute of actual execution (fractions of a cent for a few-second job), which is not literally $0 but is close enough to negligible for this project's volume — however, it still is not a documented free tier.
- **Conclusion, per the brief's explicit instruction not to assume free hosting supports hourly jobs:** this project does **not** rely on any hosting provider's built-in cron or background-worker product to drive monitoring. Instead:
  - Backend API deploys as a Render **free Web Service** (it's fine that it sleeps — see below).
  - Frontend deploys as a Render **free Static Site** (static hosting doesn't sleep).
  - Database is **MongoDB Atlas free M0 cluster**.
  - **The scheduler is a GitHub Actions workflow** (`.github/workflows/monitor.yml`) on a `cron:` schedule, calling `POST /api/monitor/run` over HTTPS once an hour. GitHub Actions is genuinely free for this volume (public repos: unlimited; private repos: 2,000 free minutes/month, and this workflow uses well under a minute per run). Because the call itself is an HTTPS request, it also **wakes** the sleeping Render web service as a side effect — solving two problems (the "who runs the schedule" problem and the "free tier sleeps" problem) with one genuinely free mechanism, rather than paying for an always-on worker.
- This is an honest trade-off, not a claim of production-grade infrastructure: GitHub Actions' cron schedule is "best effort" (GitHub documents that scheduled workflows can be delayed during high load, typically by a few minutes, rarely more) and a cold Render instance adds a few seconds of latency to the very first request after sleep. For a 4-beginner, 7-day training project, this is acceptable and is explained plainly in the demo rather than hidden.

---

13. User-interface specification

General direction: clean, modern, trustworthy — this is a tool people use to make a purchase decision, so clarity beats decoration. Price is always the most visually dominant number on any card. The cheapest valid offer in a comparison is marked with a text label ("Lowest price") in addition to any color, never color alone (brief requirement, Section 3.1).

Palette: primary `#0F6E56` (teal-green, used for primary buttons, the "lowest price" badge, and links), neutral text `#1C1C1A`, muted text `#6B6B66`, background `#F5F5F4`, card surface `#FFFFFF`, border `#E2E1DB`, success `#1D9E75`, warning/stale `#B8860B`, error `#A32D2D`, sample-data badge `#6B6B66` on `#ECECE8`. **Typography:** system font stack (`-apple-system, Segoe UI, Roboto, sans-serif` — matches Bootstrap's default, no custom font loading needed). Page titles 22–24px/600 weight, section headers 16–18px/600, body 14–15px/400, meta text 12–13px/400 muted. **Spacing scale:** 4/8/12/16/24/32px. Cards use 16px internal padding, 16–24px gaps between cards. **Components:** Bootstrap 5 cards for offer cards and dashboard tiles, Bootstrap badges for status (`In stock` / `Out of stock` / `Sample data` / `Stale`), Bootstrap forms for all inputs, Bootstrap navbar for the top nav.

13.1 Landing / home page

Purpose: explain PriceRadar in one screen and get the visitor searching or signing up. Hero section with a search bar (same component as the search page's bar, auto-focused), a one-line value proposition, and 2–3 feature highlights (compare prices, get alerts, Telegram/email notifications). Nav: logo/home, Search, Login/Register (logged out) or Dashboard/Logout (logged in). No loading/empty/error state beyond the search bar's own (Section 13.2). Responsive: hero stacks vertically under 768px.

13.2 Search page

Purpose: let the user find a product. A single search input with a submit button (and submit-on-Enter), placeholder "Search for a product, brand, or model". Below it, once a search has been run, inline results (see 13.3) — this page and the results page can be the same route (`/search?q=...`) rather than two hops. Loading: a Bootstrap spinner under the search bar. Empty: "No products matched ". Try a different search." Error: "Something went wrong searching — try again." with a retry button. Accessibility: input has a visible `<label>` (visually hidden if needed), results are in a `<ul>` / `<li>` or equivalent landmark structure.

13.3 Search-results page (same route as 13.2 after a query)

Each result is a Product card: title, brand/variant line, and up to 2 offer rows showing provider name, price (large, bold), a "Sample data" badge when `isSample`, and a "View on " link (`productUrl`, opens in a new tab). The cheapest offer in the card is marked "Lowest price". Clicking the card (not the external link) goes to the comparison/detail page (13.4). Loading/empty/error: same pattern as 13.2.

13.4 Product comparison / details page (`/products/:productId`)

Purpose: full comparison across every offer for one product. Header: product title, brand, variant, image if available. Below: one row per offer (provider, price, availability badge, last-updated timestamp formatted as "Updated 12 minutes ago" or "Stale — last checked 3 days ago" or "Error fetching latest price" matching `lastStatus`, `isSample` badge, "View on " button) sorted price-ascending, with "Lowest price" text label (not color alone) on the top valid (`in_stock` or `unknown`, not `error`) row. A primary "Add to watchlist" button per offer (requires login — if logged out, clicking prompts login and returns here after). If price history exists (stretch goal), a simple line chart below. Loading: skeleton rows. Empty: "This product isn't tracked yet." (shouldn't normally happen from a search click, but handled). Error: inline error banner, page content that did load stays visible.

13.5 Registration page

Fields: Name, Email, Password, Confirm password. Client-side validation: all required, email format, password minimum 8 characters, confirm must match — inline error text under each field, submit disabled while invalid. On success: store JWT, redirect to Dashboard. On server error (e.g. duplicate email, `409 CONFLICT`): show the server's error message under the Email field. Loading: submit button shows a spinner and is disabled during the request.

13.6 Login page

Fields: Email, Password. Same validation pattern. On `401 AUTH_INVALID`: "Email or password is incorrect" (never reveal which one). On success: store JWT, redirect to the page the user was trying to reach, or Dashboard by default.

13.7 User dashboard (`/dashboard`, requires login)

Purpose: at-a-glance summary. Three summary tiles (Watchlist count, Active alerts count, Unread/recent notifications count), then a short list of the most recent 5 notification events, then quick links to Watchlist, Alerts, Notification Preferences. Loading: skeleton tiles. Empty: "You're not tracking anything yet — search for a product to get started" with a button to Search.

13.8 Watchlist page (`/watchlist`, requires login)

List of `WatchlistItem` rows, each rendering an offer card (provider, title, price, `isSample`/status badges) with a "Remove" button and an "Add alert" button (pre-fills the offerId on the Create Alert form). Loading/empty ("Nothing on your watchlist yet")/error states as elsewhere.

13.9 Create/edit alert form (`/alerts/new?offerId=...` or `/alerts/:alertId/edit`)

Fields: read-only offer summary at top (which product/provider this alert is for), Alert type (radio: "Notify at or below a price" / "Notify on a price drop of at least"), conditionally shown numeric field (Target price in ₹ / Drop amount in ₹) with currency symbol prefix, Notification channels (checkboxes: Email, Telegram — Telegram checkbox disabled with a tooltip "Link your Telegram first" if `telegramChatId` isn't set yet, linking to Notification Preferences). Validation: exactly one of target/drop required depending on type, value must be a positive number, at least one channel selected. On success: redirect to Alert management with a success toast ("Alert created").

13.10 Alert-management page (`/alerts`, requires login)

Table/list of alerts: offer title, type + condition in plain language ("Alert when ≤ ₹14,999" / "Alert on a ₹500+ drop"), channels, status (`active` / `paused` badge), last triggered date if any, and row actions (Edit, Pause/Resume, Delete with a confirm dialog). Empty: "No

alerts yet — add one from an offer's page."

13.11 Notification preferences page (`/settings/notifications`, requires login)

Shows current email (read-only, tied to account email for this MVP — no separate notification email), Telegram link status (unlinked: shows a "Link Telegram" button that opens instructions + the bot username + a generated link code to send as `/start <code>`; linked: shows "Telegram linked ✓" with an Unlink option). Loading/success/error states for the link action specifically (e.g. "Code expired, generate a new one").

13.12 Notification history page (`/notifications`, requires login)

Table of `NotificationEvent` rows: date, which alert/offer, channel (with icon), status badge (`Sent` / `Failed` / `Skipped — duplicate`), and the message text (collapsible/truncated). Empty: "No notifications yet."

13.13 Unauthorized, not-found, and general error states

- **401/unauthorized route access:** redirect to `/login` with a message "Please log in to continue," returning to the original destination after login.
- **404 page:** simple centered message "Page not found" with a link home.
- **General/unexpected error (e.g. a crashed API call not otherwise handled):** a shared `<ErrorBanner>` component with "Something went wrong. Please try again." and a retry affordance where applicable; never a blank white screen.

---

14. Team responsibility matrix

Feature area	Primary owner	Reviewer	Depends on	Files owned	Do not edit without agreement	Integration milestone
API contract & docs	Member 1 (lead)	whole team	—	`docs/`, `README.md`	—	Day 1
Price comparison engine	Member 1	Member 3	Offer schema (3)	`server/src/services/priceComparison.js`	—	Day 4
Monitoring orchestration	Member 1	Member 4	Provider interface (4), Offer/PriceObservation schema (3)	`server/src/services/priceMonitor.js`	—	Day 4
Alert evaluation	Member 1 (coordinate with 3)	Member 3	Alert schema (3)	`server/src/services/alertEvaluator.js`	Alert model fields	Day 4
React app shell, routing, auth UI	Member 2	Member 1	API contract (Section 6)	`client/src/App.jsx`, `client/src/main.jsx`, `client/src/context/`, `client/src/pages/Login*`, `Register*`	`server/**`	Day 3
Search/comparison/dashboard/watchlist/alert UI	Member 2	Member 1	API contract, UI spec (Section 13)	`client/src/pages/`, `client/src/components/`, `client/src/services/api.js`, `client/src/styles/`	`server/**`	Day 4–5
Express app structure, DB models, auth	Member 3	Member 1	DB schema (Section 5)	`server/src/models/`, `server/src/middleware/`, `server/src/config/`, `server/src/app.js`, `server/src/server.js`	`server/src/providers/`, `server/src/services/notifications/`	Day 3
Watchlist/alert/notification-pref CRUD routes	Member 3	Member 1	DB schema, API contract	`server/src/routes/`, `server/src/controllers/`	`server/src/services/priceMonitor.js`	Day 3–4

Feature area	Primary owner	Reviewer	Depends on	Files owned	Do not edit without agreement	Integration milestone
Retailer research & provider adapters	Member 4	Member 1	Provider interface (Section 10.5)	`server/src/providers/`	`server/src/services/priceComparison.js`, `server/src/services/priceMonitor.js`	Day 1 (research), Day 2 (fixture), Day 4 (real, if approved)
Email + Telegram notification services	Member 4	Member 1	Notification requirements (Section 3.5)	`server/src/services/notifications/`	—	Day 5
GitHub repo, branches, PR process	Member 1	—	—	repo settings	—	Day 1
Deployment & scheduler wiring	Member 1	Member 4	Working app from all members	`.github/workflows/monitor.yml`, hosting dashboards	—	Day 6–7

Rule from the brief, restated: Member 1 reviews every change to shared contracts (this document, the API routes' shapes, the Alert/Offer models), auth-sensitive code, monitoring orchestration, and production configuration. Every other PR gets at least one review from a teammate who isn't the author.

---

15. Seven-day plan — milestones (full day-by-day task breakdown is in Documents 2–5; this is the cross-team summary)

Day	Objective	End-of-day acceptance check
1	Contracts finalized (this document), repo + branches live, scaffolding starts, Flipkart registration submitted	`npm run dev` starts both client and server locally with no errors; this document is merged to `main`
2	Core data + search, fixture provider working	Searching for a fixture product returns results end-to-end through the real API, not mocked in the frontend
3	Auth, dashboard, watchlist, alert CRUD	A registered user can log in, add an offer to their watchlist, and create an alert; a second user cannot see the first user's data
4	Comparison engine, monitoring orchestration, alert evaluation	`POST /api/monitor/run` (with the correct secret) processes every distinct offer once, writes observations, and a manually-lowered fixture price correctly triggers a target-price alert
5	Notifications wired end-to-end	A triggered alert produces a real email via Brevo and a real Telegram message, both recorded in `NotificationEvent`; running the same trigger twice does not send twice
6	Stabilization, deployment	Frontend, backend, and database are all live on their hosting; `/api/health` responds from the public URL
7	Final verification + demo	GitHub Actions has successfully called `/api/monitor/run` on its schedule at least once in production; the acceptance checklist (Section 17) passes; demo rehearsed

---

16. Definition of done

A feature is "done," not "working on my machine," when:

1. It matches this document's schema/API/env-var contract exactly.
2. It has at least the tests listed for it in Section 17.
3. It has been reviewed and merged via a pull request (Document 5).
4. It behaves correctly in the deployed environment, not only locally.
5. Any sample/simulated data involved is visibly labeled as such in the UI.

---

17. Acceptance tests (summary — full test matrix is in each role document)

ID	Area	Check
ACC-01	Search	Searching a known fixture product name returns it with at least 2 labeled offers
ACC-02	Comparison	The lowest valid offer is text-labeled "Lowest price," not color-only
ACC-03	Auth	A second registered user cannot GET another user's `/api/watchlist` or `/api/alerts`
ACC-04	Monitoring	A fixture price edited to drop below a target-price alert's threshold, followed by `POST /api/monitor/run`, results in exactly one `NotificationEvent` with `status: "sent"` per channel
ACC-05	Dedup	Calling `/api/monitor/run` a second time immediately after, with the price unchanged, produces no new `"sent"` events for that alert
ACC-06	Failure handling	A provider forced to fail (e.g. fixture entry temporarily marked invalid) results in `Offer.lastStatus = "error"`, not a price of 0 and not a false alert
ACC-07	Deployment	`GET /api/health` returns `200 {status:"ok"}` from the public backend URL
ACC-08	Scheduler	The GitHub Actions workflow run log shows at least one successful `POST /api/monitor/run` call against the deployed backend
ACC-09	Secrets	`git log -p -- '**/.env'` on the repo shows no `.env` file was ever committed
ACC-10	Honesty	Every `isSample: true` offer visibly shows a "Sample data" badge in the UI

---

18. Risks, assumptions, and fallback decisions

Risk	Likelihood	Impact	Fallback (already decided, not a later discussion)
Flipkart affiliate approval doesn't arrive within 7 days	Medium–High (unverified SLA, third-party reports suggest days-to-weeks)	Medium	Ship entirely on the fixture provider; demo states plainly that live Flipkart integration is pending approval, with the provider code ready to switch on
Amazon API inaccessible	Certain (verified: 10-sale eligibility gate)	Low (already excluded from scope)	Amazon only ever appears as sample data
Render free web service cold-start adds latency to the first request after idle	High (documented platform behavior)	Low	Acceptable for a training-project demo; mention it rather than hide it
GitHub Actions scheduled runs can be delayed a few minutes under load	Low-Medium (documented "best effort" behavior)	Low	Acceptable; the brief only requires honestly-described near-hourly monitoring, not a hard real-time SLA
A team member's Windows machine has Node < 20 or no Git	Medium	Medium	Day 1 setup step in every document checks `node --version` / `git --version` first, with install links
Scope creep (e.g. building price-history charts before the core flow works)	Medium	Medium	Section 1.2 explicitly orders charts after Day 5; Day 6 explicitly includes "freeze unnecessary features"

Assumptions made explicit, per the instruction not to leave resolvable decisions open: Bootstrap over a custom design system (speed), JWT in localStorage over cookies (simplicity, documented trade-off), GitHub Actions over a paid worker (the only verified $0 option), one shared Brevo account rather than 4 individual ones (avoids fragmenting the 300/day limit and delivery history).

---

19. Final release criteria

The project is releasable for the Day 7 demo when every item in Section 17's acceptance tests passes **against the deployed URLs**, not just locally, and when the team can truthfully state, for each data source shown in the UI, whether it is live or sample — with no exceptions to that honesty rule even under demo-day pressure.